

Neural Models

Depth, activations, and the output layer

Tirtharaj Dash

Department of CS & IS
BITS Pilani, Goa Campus, India
[Homepage](#)

August 2026

So far:

In the last lecture, we discussed learning as optimisation of a neural model.

To that end, we looked at a single hidden-layered neural net with a single output, and showed how backprop reduces loss.

Loss	$\frac{1}{2}(\hat{y} - y)^2$ (reduced)
Method	partial derivative, backward sweep
Tools	the chain rule

But our network was a toy deep network with no justification of its structure.

Stacking layers

We now write the network as a stack of L layers, with $\mathbf{h}^{(0)} = \mathbf{x}$:

$$\begin{aligned}\mathbf{a}^{(\ell)} &= W^{(\ell)} \mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \\ \mathbf{h}^{(\ell)} &= f^{(\ell)}(\mathbf{a}^{(\ell)}).\end{aligned}$$

The recursion already defines the complete composition:

$$F(\mathbf{x}; \theta) = \mathbf{h}^{(L)}, \quad \theta = \left\{ W^{(\ell)}, \mathbf{b}^{(\ell)} \right\}_{\ell=1}^L.$$

Our toy network was this with $L = 2$; and backprop doesn't care whether L is small or large.

Backpropagation at any depth

Let $\delta^{(\ell)} = \partial \mathcal{L} / \partial \mathbf{a}^{(\ell)}$, the gradient at layer ℓ 's pre-activations. For elementwise activations,

$$\delta^{(L)} = \nabla_{\hat{\mathbf{y}}} \mathcal{L} \odot f'(\mathbf{a}^{(L)})$$

$$\delta^{(\ell)} = \left(W^{(\ell+1)\top} \delta^{(\ell+1)} \right) \odot f'(\mathbf{a}^{(\ell)})$$

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \delta^{(\ell)} \left(\mathbf{h}^{(\ell-1)} \right)^\top$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \delta^{(\ell)}$$

Steps B1 to B6 (last lecture) were these four lines with $L = 2$ written out in full. The second line is the recursion: it carries δ one layer down and does the whole job.

For a vector-valued output activation, use its Jacobian. In particular, softmax with cross-entropy gives $\delta^{(L)} = \mathbf{p} - \mathbf{y}$.

Nonlinearity of f

Suppose we drop f and keep only the affine parts. Then

$$W^{(L)} \left(\dots W^{(2)} \left(W^{(1)} \mathbf{x} + \mathbf{b}^{(1)} \right) + \mathbf{b}^{(2)} \dots \right) + \mathbf{b}^{(L)} = W_{\text{eff}} \mathbf{x} + \mathbf{b}_{\text{eff}}$$

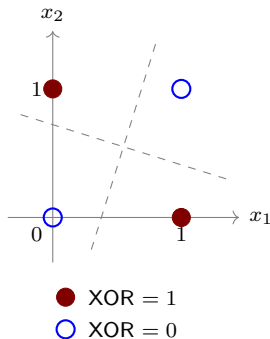
for one effective matrix and one effective bias.

A stack of affine layers is one affine model wearing a costume.

That is, if we have linear activation functions at hidden layers, L will not matter – It will remain a single layered network, effectively.

The XOR problem

No straight line separates the two classes.



The XOR problem

Example: with a step activation, two hidden units are enough.

$$h_1 = \text{step}(x_1 + x_2 - 0.5), \quad h_2 = \text{step}(x_1 + x_2 - 1.5)$$

$$\hat{y} = \text{step}(h_1 - h_2 - 0.5)$$

x_1	x_2	h_1	h_2	$\hat{y}$	XOR
0	0	0	0	0	0
0	1	1	0	1	1
1	0	1	0	1	1
1	1	1	1	0	0

h_1 computes OR and h_2 computes AND. Here we chose these features by hand; during training, hidden units can discover analogous intermediate features.

Minsky & Papert, *Perceptrons*, MIT Press, 1969

Activation functions

Func	$f(z)$	$f'(z)$	Note
Sigmoid	$1/(1 + e^{-z})$	$\sigma(z)(1 - \sigma(z))$	peaks at 0.25
Tanh	$\tanh z$	$1 - \tanh^2 z$	peaks at 1, zero-centred
ReLU	$\max(0, z)$	1 if $z > 0$, 0 if $z < 0$	active derivative is 1
Leaky ReLU	$\max(\alpha z, z)$	1 or α	nonzero for $z < 0$
GELU	$z \Phi(z)$	$\Phi(z) + z\phi(z)$	common in Transformers

At $z = 0$, ReLU and leaky ReLU use an implementation-dependent subgradient convention. At $z = 0$ neither has a derivative: the slope is 0 from the left and 1 from the right. Any value in between is a valid subgradient, and implementations simply pick one — PyTorch returns 0. Landing exactly on $z = 0$ is rare enough, and will not show up in training.

Why ReLU helped deep networks

Sigmoid contributes a derivative factor of at most 0.25 componentwise, and much less when a unit saturates. The weight matrices are unbounded, though: a matrix with large singular values stretches whatever passes through it. So the product can decay or grow, depending on which factor dominates.

ReLU contributes a derivative factor of 1 on an active unit, although the weight matrix still transforms the gradient. The cost is that inactive units pass zero; a permanently negative unit can stop learning.

Leaky variants reduce this “dying ReLU” problem.

Vanishing and exploding gradients

Let $D^{(\ell)} = \text{diag}(f'(\mathbf{a}^{(\ell)}))$. Writing the recursion in its correct order gives

$$\boldsymbol{\delta}^{(1)} = D^{(1)}W^{(2)\top}D^{(2)}W^{(3)\top} \dots D^{(L-1)}W^{(L)\top}\boldsymbol{\delta}^{(L)},$$

and $\partial\mathcal{L}/\partial W^{(1)} = \boldsymbol{\delta}^{(1)}\mathbf{h}^{(0)\top}$.

Jacobians mostly contract gradient norms tend to shrink

Jacobians strongly expand gradient norms can grow rapidly

Long products can therefore become badly conditioned. This is why singular values, non-saturating activations, careful initialisation and skip connections matter when training deep networks.

Glorot & Bengio, *AISTATS*, 2010; He et al., *CVPR*, 2016

Why we do not start from zero

Set every weight and bias in our last lecture's network to zero and train it.

You will notice that the hidden units behave identically at every step. With the same example and $\eta = 1$:

Step	$\mathbf{h}$	rows of $W^{(1)}$ equal?
0	(0.5000, 0.5000)	yes
1	(0.5000, 0.5000)	yes
2	(0.5010, 0.5010)	yes
3	(0.5027, 0.5027)	yes

Both units see the same input, compute the same output and receive the same gradient, so they stay equal forever. Two hidden units, one distinct function between them.

Why we do not start from zero

Random weight initialisation breaks the symmetry; hidden biases may still start at zero. The weight scale matters separately because it affects the layer Jacobians.

Glorot & Bengio, *AISTATS*, 2010; He et al., *ICCV*, 2015

The output layer is task-dependent

The hidden layers are generic. The last layer and the loss are chosen together, to match what is being predicted (the task at hand).

Task	Output layer	Loss
Real number	linear	squared error
One yes/no answer	sigmoid	binary cross-entropy
One of K classes	softmax	cross-entropy

A classifier should output a distribution over K classes, so the K numbers must be positive and sum to one. Softmax does exactly that:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

The raw scores z_k are called logits.

Exponentiating makes everything positive and keeps the ordering (also adds a bit of smoothness); dividing by the sum normalises. Adding a constant to every logit changes nothing, so only the differences between logits carry information.

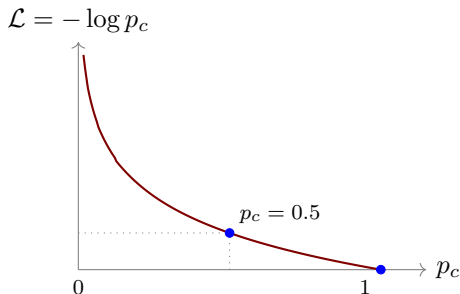
In implementations, subtract $\max_j z_j$ before exponentiating; this leaves the probabilities unchanged and avoids numerical overflow.

Cross-entropy

With a one-hot target $\mathbf{y}$ marking the correct class c ,

$$\mathcal{L} = - \sum_{k=1}^K y_k \log p_k = -\log p_c$$

Only the probability assigned to the correct class appears explicitly, but p_c depends on every logit through the softmax denominator.

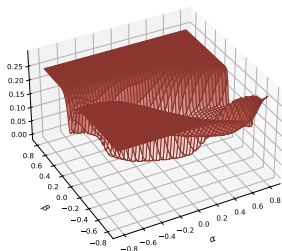


Cross-entropy

$$\begin{aligned} p_c \rightarrow 1 & \quad \mathcal{L} \rightarrow 0 \\ p_c \rightarrow 0 & \quad \mathcal{L} \rightarrow \infty \end{aligned}$$

The loss punishes confident mistakes very heavily, which is what we want from a model that outputs probabilities.

Loss surface of our 2-2-1 MLP



```
d1 = rng.normal(size=9)
d1 *= norm(th)/norm(d1)
```

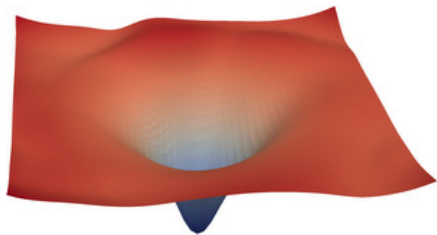
```
d2 = rng.normal(size=9)
d2 -= d2@d1/(d1@d1)*d1
d2 *= norm(th)/norm(d2)
```

```
Z = [[loss(th + a*d1 + b*d2)
      for b in grid]
      for a in grid]
```

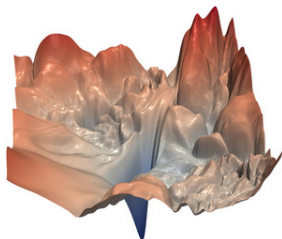
Our 2-2-1 network trained on XOR, sliced along two random directions in its nine-dimensional weight space. The centre is the trained solution, and one unit of α perturbs every weight by about its own size. The flat shelf is where the sigmoids have saturated.

Loss surface at scale

ResNet-56 on CIFAR-10, sliced the same way in both panels. The only difference is whether the skip connections are present.



with skip connections



without them

Remove the skips and the surface turns chaotic. That is a second reason they help, alongside keeping the gradient alive.

Li et al., *NeurIPS*, 2018

Homework (write or code)

Same network as last lecture, with three output units instead of one. The hidden layer is untouched, so $\mathbf{h} = (0.6225, 0.3430)$ as before.

$$W^{(2)} = \begin{bmatrix} 0.3 & -0.5 \\ -0.2 & 0.4 \\ 0.1 & 0.2 \end{bmatrix}, \quad \mathbf{b}^{(2)} = \begin{bmatrix} 0.2 \\ 0.0 \\ -0.1 \end{bmatrix}$$

$$\mathbf{z} = W^{(2)}\mathbf{h} + \mathbf{b}^{(2)} = (0.2152, 0.0127, 0.0308)$$

$$\mathbf{p} = \text{softmax}(\mathbf{z}) = (0.3776, 0.3084, 0.3140)$$

With the true class being the first, $\mathcal{L} = -\log 0.3776 = 0.9739$.

Homework (write or code)

Softmax and cross-entropy differentiated together collapse to one line:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{z}} &= \mathbf{p} - \mathbf{y} = (0.3776 - 1, 0.3084, 0.3140) \\ &= (-0.6224, 0.3084, 0.3140)\end{aligned}$$

Checking against finite differences on each logit:

	analytic	finite difference
$\partial \mathcal{L} / \partial z_1$	-0.62239	-0.62239
$\partial \mathcal{L} / \partial z_2$	0.30837	0.30837
$\partial \mathcal{L} / \partial z_3$	0.31402	0.31402

Homework (write or code)

From here the recursion takes over unchanged:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W^{(2)}} &= (\mathbf{p} - \mathbf{y}) \mathbf{h}^\top \\ \boldsymbol{\delta}^{(1)} &= \left(W^{(2)\top} (\mathbf{p} - \mathbf{y}) \right) \odot f'(\mathbf{a}^{(1)})\end{aligned}$$

Reading $\mathbf{p} - \mathbf{y}$

The gradient at the logits is the predicted distribution minus the true one.

Correct class	$p_c - 1$, negative, so push its logit up
Every other class	p_k , positive, so push its logit down
Confident and right	all components near zero, so barely move

An analogous cancellation occurs for a sigmoid output with binary cross-entropy:

$$\frac{\partial \mathcal{L}}{\partial z} = \hat{y} - y.$$

Last lecture instead used sigmoid with squared error, for which $\partial \mathcal{L} / \partial z = (\hat{y} - y) \sigma'(z) = -0.1103$. That objective is valid, but the standard likelihood-based pairings give the simpler logit gradient.

Goodfellow et al., *Deep Learning*, Ch. 6.2

From one example to a minibatch

Everything so far used a single example. A dataset needs an average:

$$\mathcal{L}(\theta) = \frac{1}{|B|} \sum_{i \in B} \mathcal{L}^{(i)}(\theta) \quad \implies \quad \nabla_{\theta} \mathcal{L} = \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \mathcal{L}^{(i)}$$

Differentiation is linear, so the gradient of the average is the average of the gradients. Nothing in the backward pass changes.

In practice B is a minibatch, a few dozen to a few thousand examples. It is large enough for the average to be a usable estimate, and small enough that the cached activations fit in memory.

Connection to LLMs

Take the third row of the output table and let K be the size of a vocabulary rather than a handful of classes.

	Today	A language model
Classes K	3	often tens of thousands of tokens
Input	one vector	a sequence of tokens
Target	the labelled class	the token that came next
Predictions	one	one at every position

Predicting the next token is classification over the vocabulary.
Softmax, cross-entropy and $\mathbf{p} - \mathbf{y}$ all carry over unchanged.

Connection to LLMs

Stays the same

softmax over the outputs

cross-entropy as the loss

$\mathbf{p} - \mathbf{y}$ at the logits

the δ recursion

Changes

how the input is represented

what sits between input and logits

how many predictions per example

the scale of everything

The right-hand column is what we build over the next two weeks. The left-hand column is the computational core that carries over unchanged.

Summary

- ▶ Depth is a composition of layers, and one recursion differentiates all of them.
- ▶ Without a nonlinearity the whole stack collapses to one affine map.
- ▶ Activations are chosen for both representational and optimisation properties.
- ▶ Identical weight initialisation leaves hidden units identical forever.
- ▶ The output layer and the loss are picked together to match the task.